Salida renderizada en lidr_4

Prompt v4, RenderedResult y POST /api/v1/estimate/rendered · continuación de «Arquitectura de lidr_4» · rama entrega/lidr_4 · 6 de octubre de 2026

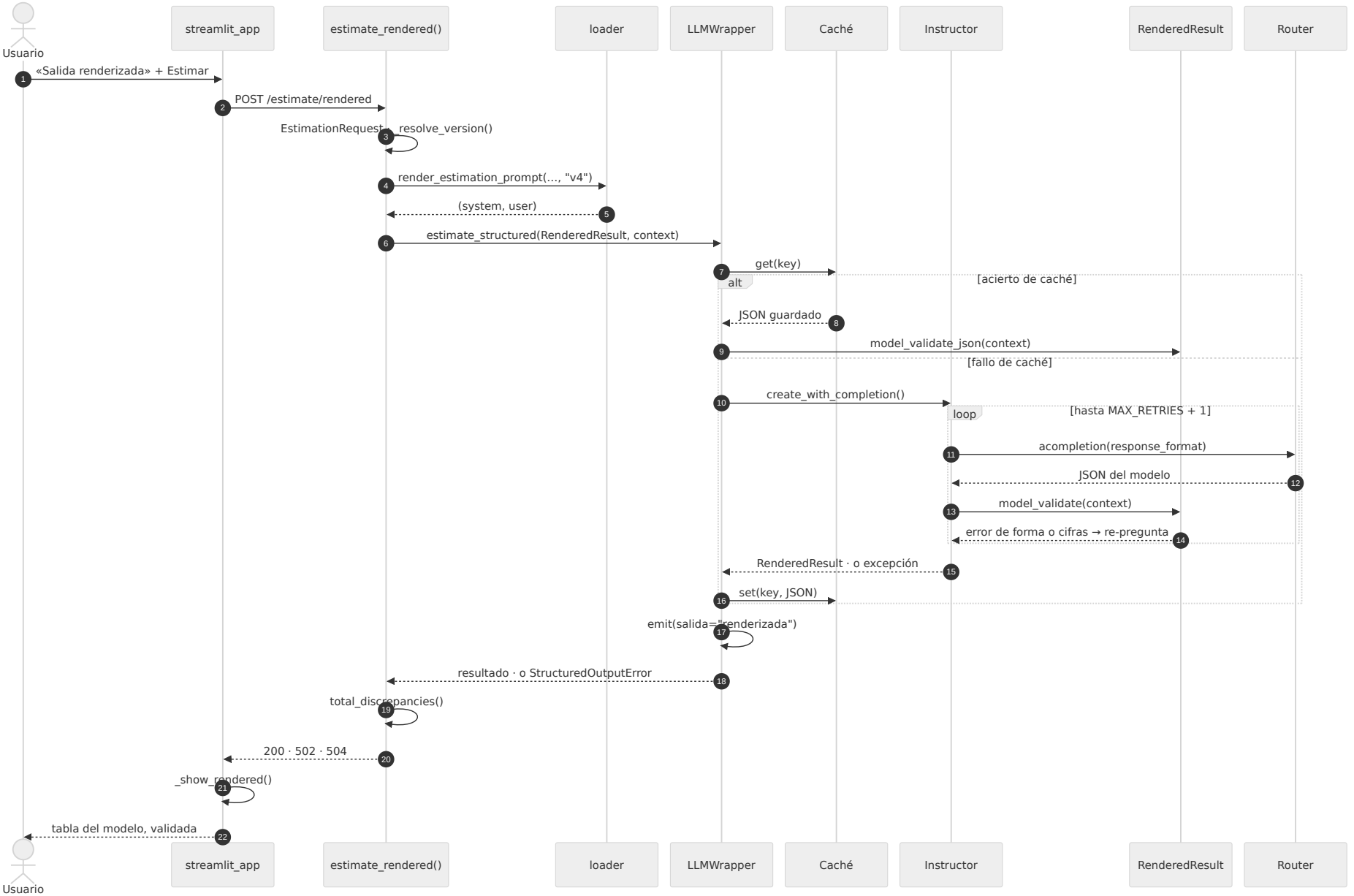
1 · Capas y llamadas

Cada fila es una capa y cada caja, un módulo, una clase o un endpoint con sus funciones. Entre capas, las llamadas de una estimación renderizada. Borde azul: nuevo o cambiado para la salida renderizada.

1 · Frontend streamlit_app.py	main() st.form + 3 selectbox · selector de versión · 2 interruptores	_switch_off_other() · _output_mode() interruptores excluyentes → text · structured · rendered	_estimate() · _estimate_structured() · _estimate_rendered() httpx.Client.post al endpoint del modo	_show_structured() · _show_rendered() _show_metrics() · _show_team_and_risks()
↓ POST /api/v1/estimate/rendered (o /structured, o /estimate) · GET /health				
2 · API app/main.py · app/routers/estimations.py	GET /health prompt_versions · structured_... · rendered_prompt_versions	POST /api/v1/estimate estimate() → texto	POST /api/v1/estimate/structured · /rendered estimate_structured() · estimate_rendered()	_estimate_validated(request, version, wrapper, response_model, context) _resolve_version() · _configured_version() · _log_failure()
↓ FastAPI valida → EstimationRequest · render_estimation_prompt(request, "v4") · await estimate_structured(…, RenderedResult, context)				
3 · Esquemas app/schemas/	EstimationRequest description · project_type · detail_level · output_format	StructuredResult phases · team · totals · summary · confidence_pct · total_discrepancies()	RenderedResult(StructuredResult) + rendered · _rendered_matches_format(info.context)	…Response EstimationResponse · StructuredEstimationResponse · RenderedEstimationResponse
4 · Prompts app/prompts/loader.py · estimation/vN/	render_estimation_prompt(request, version) → (system, user) · filtros json y number	available_versions(output) · output_of_version() text · structured · rendered (clave output de examples.yaml)	v1 · v2 · v3 texto · texto · estructurada	v4 estructurada + rendered según output_format
5 · Servicios app/services/	LLMWrapper.estimate() texto libre	LLMWrapper.estimate_structured(…, response_model, context) caché · coste de todos los intentos · emit()	Instructor from_litellm(router.acompletion, JSON_SCHEMA) · create_with_completion(context=...)	EstimationCache make_key() · get() · set() (valida con el mismo context)
↓ acompletion() con el schema como response_format · validación con context · get/set en Redis · emit()				
6 · Configuración e infraestructura app/config.py · app/cache.py · app/tracing.py	Settings · get_settings() PROMPT_VERSION · STRUCTURED_PROMPT_VERSION · RENDERED_PROMPT_VERSION · STRUCTURED_MAX_RETRIES	create_cache_client() · get/set_cached_estimation() fail soft	emit(logger, evento, **campos) estimacion_completada (salida=renderizada, intentos)	
7 · Externos	LiteLLM Router acompletion() · completion_cost() · primario + respaldo	OpenAI / Anthropic	Redis	

2 · Secuencia de una estimación renderizada

Recorrido de un envío con «Salida renderizada». El validador de `RenderedResult` recibe `output_format` en el contexto: si la forma o las cifras de `rendered` no coinciden, Instructor re-pregunta con el error; agotados los intentos, 502.



3 · Tres salidas, un mismo formulario

El prompt es un artefacto compuesto: estructura fija (las plantillas `.j2`), variables de la request (la descripción) y parámetros del usuario (tipo, nivel de detalle y formato). Lo que cambia entre las tres salidas es dónde termina `output_format`.

	Texto libre	**Salida estructurada**	**Salida renderizada (nuevo)**
Interruptores	ninguno	«Salida estructurada»	«Salida renderizada» (excluyente con el otro)
Endpoint	`POST /api/v1/estimate`	`POST /api/v1/estimate/structured`	`POST /api/v1/estimate/rendered`
Versiones	`v1`, `v2` (`PROMPT_VERSION`)	`v3` (`STRUCTURED_PROMPT_VERSION`)	`v4` (`RENDERED_PROMPT_VERSION`)
Respuesta del LLM	Texto Markdown	JSON que cumple `StructuredResult`	JSON que cumple `RenderedResult`: el de `v3` más `rendered`
`output_format`	Entra al prompt	No entra al prompt	Entra al prompt y al validador (contexto de Instructor)
Quién presenta	El modelo, sin validar	El frontend, en código	El modelo, en `rendered`, validado contra las cifras
Si falla	—	Re-pregunta; 502 al agotar intentos	Re-pregunta también por la forma o las cifras de `rendered`; 502

La contradicción de session_4_live

Su `system.j2` pide la respuesta entera como tabla Markdown, lista o prosa, y en el mismo prompt exige un objeto estructurado. Las dos instrucciones compiten por el mismo lugar; gana el schema, porque Instructor lo impone, y la presentación se pierde. El autor de la referencia confirmó el diagnóstico y recomendó separar ambas cosas: que el LLM devuelva siempre la estructura y que la presentación la arme el backend o el frontend. Eso es lo que hace `v3`.

Por qué existe v4

`v4` resuelve la misma contradicción por otro camino: el texto va en un campo propio, `rendered`, al final del schema, así que no compite con él. A cambio, el servicio lo valida: el formato pedido y las mismas horas y costes que los campos. Es un guardrail de salida: `output_format` cambia el texto, nunca las cifras.

Costo: más tokens por respuesta y el riesgo, aún sin medir con el modelo real, de que el validador rechace cifras escritas de otra forma (PLAN §9). `v3` y `v4` conviven.

4 · Del formulario a la presentación validada: un ejemplo

La misma elección que en el reporte anterior, ahora con «Salida renderizada» y el prompt `v4`.

Tipo de proyecto: **SaaS web** → `web_saas` | Nivel de detalle: **Detallado** → `detailed` | Formato de salida: **Tabla de fases** → `phases_table`

Salida: **renderizada** → `/api/v1/estimate/rendered` | Versión del prompt: `v4`

Paso 1 · Dos interruptores excluyentes (streamlit_app.py)

Cada interruptor tiene un callback que apaga al otro. El modo resultante decide el endpoint y qué versiones ofrece el selector, que salen de `/health`.

Con los dos apagados la salida es texto libre, como antes. El formulario (los tres `selectbox` y `_build_payload()`) no cambió.

```
STRUCTURED_TOGGLE = "structured_output"
RENDERED_TOGGLE = "rendered_output"

def _switch_off_other(state, active: str, other: str) -> None:
    """Callback de un interruptor: si quedó encendido, apaga el otro."""
    if state.get(active):
        state[other] = False

def _output_mode(state) -> str:
    if state.get(RENDERED_TOGGLE):
        return "rendered"
    if state.get(STRUCTURED_TOGGLE):
        return "structured"
    return "text"

# En main():
st.sidebar.toggle("Salida estructurada", key=STRUCTURED_TOGGLE,
                  on_change=_switch_off_other,
                  args=(st.session_state, STRUCTURED_TOGGLE, RENDERED_TOGGLE))
st.sidebar.toggle("Salida renderizada", key=RENDERED_TOGGLE,
                  on_change=_switch_off_other,
                  args=(st.session_state, RENDERED_TOGGLE, STRUCTURED_TOGGLE))
mode = _output_mode(st.session_state)          # "rendered"
options = _rendered_version_options(health)   # [("Predeterminada (v4)", None), ("v4", "v4")]
```

Paso 2 · El payload viaja a /estimate/rendered (streamlit_app.py)

El cuerpo es el mismo de siempre: el contrato de entrada no cambia entre las tres salidas.

```
# Mismo formulario y mismo _build_payload() que antes; cambia el endpoint:
estimate = _estimate_rendered if mode == "rendered" else _estimate_structured
estimation, prompt_version, warnings, cached = estimate(
    api_base, payload, prompt_version=chosen_prompt_version
)

def _estimate_rendered(api_base, payload, client=None, prompt_version=None):
    return _estimate_structured(api_base, payload, client,
                                prompt_version=prompt_version,
                                path="/api/v1/estimate/rendered")
```

Lo que llega a la API:

```
POST /api/v1/estimate/rendered
Content-Type: application/json

{
  "description": "Plataforma SaaS de reservas para clínicas: agenda online,
pagos con tarjeta, recordatorios por e-mail y panel de administración.",
  "project_type": "web_saas",
  "detail_level": "detailed",
  "output_format": "phases_table"
}
```

Paso 3 · FastAPI lo convierte en EstimationRequest y el endpoint elige el schema (app/routers/estimations.py)

```
@router.post("/estimate/rendered",
             response_model=RenderedEstimationResponse, ...)
async def estimate_rendered(
    request: EstimationRequest,            # ← FastAPI valida el JSON acá
    prompt_version: str | None = Query(default=None, ...),
    llm_wrapper=Depends(get_llm_wrapper),
) -> RenderedEstimationResponse:
    version = _resolve_version(prompt_version, OUTPUT_RENDERED)   # "v4"
    result, warnings = await _estimate_validated(
        request, version, llm_wrapper,
        RenderedResult,                    # schema exigido al modelo
        output_label="renderizada",
        # {"output_format": "phases_table"} para el validador:
        context={"output_format": request.output_format.value},
    )
    return RenderedEstimationResponse(
        estimation=result.estimation, ..., warnings=warnings)
```

Objeto resultante (salida real de `repr()`):

```
EstimationRequest(
    description='Plataforma SaaS de reservas para clínicas: agenda online, ...',
    project_type=<ProjectType.WEB_SAAS: 'web_saas'>,
    detail_level=<DetailLevel.DETAILED: 'detailed'>,
    output_format=<OutputFormat.PHASES_TABLE: 'phases_table'>,
)
```

Cada endpoint acepta solo las versiones de su tipo y dice dónde pedir las otras (salida real):

```
POST /api/v1/estimate/rendered?prompt_version=v3   →  422
{"detail": "La versión de prompt 'v3' es de salida estructurada: pedila en POST
 /api/v1/estimate/structured. Versiones disponibles en este endpoint: v4."}
```

Paso 4 · output_format entra al prompt v4 (app/prompts/estimation/v4/system.j2)

El loader pasa el `.value` de cada enum, como antes. Lo nuevo es que en `v4` el bloque `<output_format>` pide la presentación **dentro del campo `rendered`**, no como respuesta entera.

```
{# app/prompts/estimation/v4/system.j2 #}
- rendered: la estimación presentada en Markdown con el formato que se indica
abajo.

El usuario pidió output_format = "{{ output_format }}". En rendered:
{% if output_format == "phases_table" %}
- Escribe una tabla Markdown con las columnas Fase | Semanas | Horas | ...
{% elif output_format == "line_items" %}
- Escribe una lista numerada (1., 2., …) con una partida por fase, ...
{% elif output_format == "narrative" %}
- Escribe prosa: un párrafo por fase, ...
{% endif %}
```

Los ejemplos del prompt traen su `rendered` en el formato pedido, armado con el filtro `number` (la misma función que usa el validador). Primer ejemplo, salida real:

```
| Fase | Semanas | Horas | Coste (EUR) | Confianza (%) |
|---|---|---|---|---|
| Descubrimiento | 1 | 70 | 3.950 | 85 |
| Diseño | 1 | 70 | 3.950 | 80 |
| Implementación | 5 | 320 | 20.000 | 70 |
| QA | 1 | 35 | 2.200 | 75 |
| Lanzamiento | 1 | 70 | 3.950 | 80 |
| **Total** | 9 | 565 | 34.050 | 75 |
```

Fragmento del prompt de sistema renderizado para esta elección (salida real):

```
El usuario pidió output_format = "phases_table". En rendered:
- Escribe una tabla Markdown con las columnas Fase | Semanas | Horas | Coste
(EUR) |
  Confianza (%), la fila separadora |---| y una fila por fase, en el orden de
phases.
- Termina con una fila **Total** con los valores de totals y la confianza
global.
- Solo la tabla: sin texto antes ni después.
- Las cifras de rendered son exactamente las de los campos (phases y totals),
escritas
  en formato castellano: 29.850, no 29850,00 ni 29,850. Si cambias una cifra,
  cámbiala en los dos lugares.
- Los supuestos y los riesgos van en sus campos (assumptions, risks), no en
rendered.
```

El resto del prompt es el de `v3`; lo verifica `test_prompts_v4.py`.

Paso 5 · Instructor exige RenderedResult y le pasa el contexto al validador (app/services/llm_wrapper.py · app/schemas/structured_estimation.py)

```
# app/services/llm_wrapper.py · estimate_structured()
client = instructor.from_litellm(self._router.acompletion,
                                 mode=instructor.Mode.JSON_SCHEMA)
client.on("completion:response", responses.append)   # coste de cada intento
estimation, response = await client.chat.completions.create_with_completion(
    model=self._primary_model,
    messages=[{"role": "system", "content": system_prompt},
              {"role": "user", "content": user_prompt}],
    response_model=response_model,        # RenderedResult
    context=context,                      # {"output_format": "phases_table"}
    max_retries=self._structured_max_retries,
    max_tokens=max_tokens or self._max_tokens,
    temperature=0.1,
)
```

El schema viaja como `response_format`; `rendered` va último (salida real de `model_json_schema()`):

```
"rendered": {"type": "string", "minLength": 10, "maxLength": 6000,
  "description": "La estimación presentada en Markdown con el formato que pide
   el prompt (tabla, partidas numeradas o prosa), con las mismas cifras que
   los campos."}
properties (todas required): phases · team · totals · summary
                           · confidence_pct · rendered
```

```
class RenderedResult(StructuredResult):
    rendered: str = Field(min_length=10, max_length=6000, description=...)

    @model_validator(mode="after")
    def _rendered_matches_format(self, info: ValidationInfo) -> RenderedResult:
        output_format = (info.context or {}).get("output_format")
        if output_format is None:
            return self                    # sin contexto: solo los campos
        lines = self.rendered.splitlines()
        if self.out_of_scope:
            ...                            # solo texto, con «Fuera de alcance:»
        if output_format == "phases_table":
            blocks = self._table_rows(lines)        # cabecera, |---| y filas
        elif output_format == "line_items":
            blocks = self._numbered_items(lines)    # 1., 2., …
        else:
            blocks = self._paragraphs(lines)        # sin tablas ni listas
        missing = [p.name for p in self.phases if not any(
            p.name.casefold() in b.casefold()
            and _mentions(b, p.hours) and _mentions(b, p.cost_eur)
            for b in blocks)]
        if missing:
            raise ValueError(...)    # Instructor re-pregunta con este texto
        ...                          # y los totales tienen que aparecer
        return self
```

Mensajes que recibe el modelo al re-preguntar (salida real):

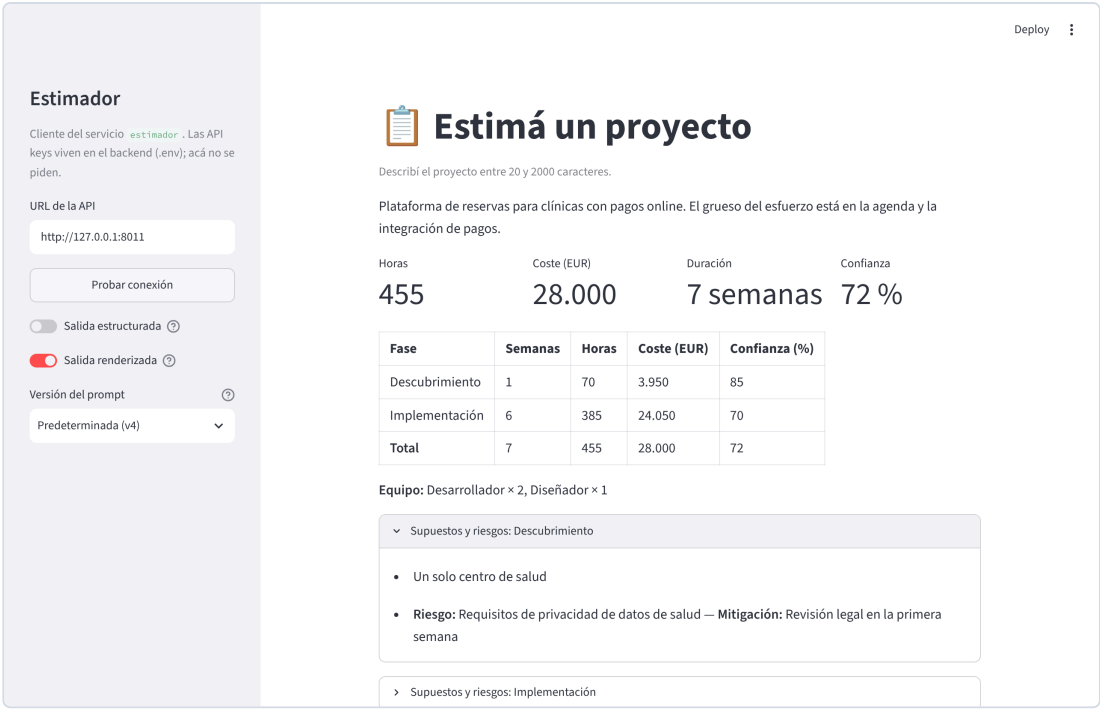
```
# 24.000 en la fila de Implementación (los campos dicen 24050):
En rendered, cada fase necesita una fila de la tabla con su nombre, sus
horas y su coste, con las mismas cifras que phases; falta o no coincide:
Implementación.

# una lista numerada cuando se pidió phases_table:
Con output_format phases_table, rendered es una tabla Markdown: fila de
cabecera, fila separadora (|---|) y una fila por fase.
```

Paso 6 · La respuesta y lo que ve el usuario

Respuesta de la API (abreviada):

```
{
  "estimation": {
    "phases": [
      {"name": "Descubrimiento", "hours": 70, "cost_eur": 3950.0, ...,
       "assumptions": ["Un solo centro de salud"],
       "risks": [{"risk": "Requisitos de privacidad de datos de salud",
                  "mitigation": "Revisión legal en la primera
semana"}]},
      {"name": "Implementación", "hours": 385, "cost_eur": 24050.0,
...}
    ],
    "team": [{"role": "Desarrollador", "headcount": 2}, ...],
    "totals": {"hours": 455, "cost_eur": 28000.0, "duration_weeks":
7.0},
    "summary": "Plataforma de reservas para clínicas con pagos online.
...",
    "confidence_pct": 72,
    "rendered": "| Fase | Semanas | Horas | Coste (EUR) | ... |\n|---
|...
                 | **Total** | 7 | 455 | 28.000 | 72 |"
  },
  "prompt_version": "v4", "cached": false, "warnings": []
}
```



`_show_rendered()` muestra el resumen, las métricas, el `rendered` del modelo tal cual (`st.markdown`), el equipo y los supuestos y riesgos por fase. Captura con el proveedor simulado de los tests (`mock_response`).

Conclusión

- `output_format` es un parámetro del prompt como el nivel de detalle; en `v4` decide qué texto escribe el modelo, y el validador lo usa para comprobarlo.
- El texto vive en un campo propio, así que no compite con el schema: es la diferencia con `session_4_live`.
- Las cifras siguen saliendo de los campos tipados; `rendered` solo puede repetirlas. Un total mal sumado sigue siendo un aviso.
- Pendiente: medir `v4` con el modelo real en los 9 casos (formato × nivel de detalle) y contar los reintentos.